

Kundsegmentering med RFM och K-Means

Projektuppgift del 2

Amir Hamza Jafari, Andreas Malmgren och Robin Hellgren

GitHub: <https://github.com/amirhamza247/ai-customer-segmentation>

1. Bakgrund

Vi har byggt en webbapp som delar in ett företags kunder i grupper utifrån hur de handlar. Då kan företaget till exempel se vilka kunder som ger mest pengar och vilka som har slutat handla.

Vi använder datasetet Online Retail II från Kaggle. Det innehåller alla köp i en brittisk webbutik från december 2009 till december 2011, drygt en miljon rader.

Varje kund beskrivs med tre värden, så kallad RFM:

- Recency: hur många dagar sedan kunden senast handlade.
- Frequency: hur många gånger kunden har handlat.
- Monetary: hur mycket kunden har handlat för totalt.

Sedan grupperar en algoritm som heter K-Means kunder som liknar varandra.

2. Så fungerar appen

Man laddar upp en CSV-fil med köp, och appen gör följande steg:

1. Rensar datan. Rader utan kund-ID, makulerade köp och rader med felaktigt pris eller antal tas bort.
2. Räknar ut RFM för varje kund.
3. Förbereder värdena så att ett fåtal kunder som handlat för enorma summor inte styr hela resultatet.
4. Föreslår hur många grupper kunderna ska delas in i. Användaren väljer 2, 3 eller 4.
5. Delar in kunderna i grupper och visar en tabell med genomsnittet för varje grupp.
6. Visar kunderna i ett diagram, färgade efter grupp.
7. Kan låta en AI-modell förklara grupperna i vanlig text. Den får bara se sammanfattningen, aldrig enskilda kunder.

3. Resultat

Efter rensningen var 805 549 av 1 067 371 rader kvar, fördelade på 5 878 kunder. De flesta borttagna raderna saknade kund-ID.

Med tre grupper blev resultatet:

- 29 % av kunderna handlar ofta, mycket och nyligen. De står för 83 % av intäkterna.
- 40 % av kunderna handlar fortfarande, men sällan och för lite. De står för 11 % av intäkterna.
- 31 % av kunderna har inte handlat på över ett år. De står för 6 % av intäkterna.

Slutsatsen är att knappt en tredjedel av kunderna står för nästan alla intäkter. Den mellersta gruppen är intressantast att satsa på, eftersom de fortfarande handlar men skulle kunna handla mer.

Grupperna går att skilja åt, men gränserna är inte skarpa. Därför har vi inte gett dem namn som "VIP", eftersom det skulle låta säkrare än det är.

4. Teknik

Appen är skriven i Python och består av tre filer:

- main.py: det användaren ser, byggt med Streamlit.
- segmentation.py: rensning, RFM och gruppering, med pandas och scikit-learn.
- ai_analysis.py: skickar sammanfattningen till AI-tjänsten Groq och hämtar förklaringen.

I första versionen sparade vi datan i en SQLite-databas. När vi bytte till att användaren laddar upp sin egen fil tog vi bort den. Varje fil analyseras för sig, så det fanns inget att spara, och databasen gjorde bara flödet längre och krångligare.

Vi har skrivit mycket av koden med hjälp av AI-kodagenter (Claude Code och Codex). I filen AGENTS.md står regler för agenten, till exempel att koden ska vara så enkel som möjligt och att den ska förklara vad den gör.

5. Utvärdering

Det gick bra att snabbt få hela flödet att fungera, från uppladdning till färdiga grupper. Vi byggde ett steg i taget och såg till att varje steg fungerade innan vi gick vidare.

Vi har lärt oss hur man förbereder data för gruppering, att gruppernas nummer inte betyder något förrän man tittar på siffrorna, och hur man styr en AI-agent så att koden inte blir onödigt stor.

På GitHub hade varje person egna brancher som slogs ihop via pull requests. Det som fungerade sämre var att vi byggde två versioner av appen samtidigt, en med databas och en med uppladdning, och fick lägga tid på att slå ihop dem.

Nästa gång skulle vi bestämma upplägget tillsammans innan vi börjar koda, använda code review mer och lägga ut appen på nätet så att den har en egen länk.